ShaderToy Sender

Version 2.010

spout.zeal.co

ShaderToySender is an application for Microsoft Windows that can be used to load shader files produced from "*ShaderToy*" and "*GLSL Sandbox*". The graphics generated by the shaders can be imported into any program that has Spout input.

Files can be created by copy/paste of shader code and saving as text files. Shaders can be saved as either 32bit or 64bit FreeframeGL 1.6 or FreeframeGL 2.0 plugins directly from the menu, so you can subsequently use them with a Freeframe Host such as Resolume, Magic or Isadora.

Installation

Unzip the distribution file and run the installer. *ShaderToySender* is built with Visual Studio 2017 and the necessary runtime dll files are included with the distribution. However, if you receive any message that a runtime dll is not available you can download and run the Microsoft runtime installers. You should only need to install the 32 bit version. But if you install both 64bit and 32bit, make sure you install the 64bit version first.

After installation, an icon will be on your desktop for "*ShaderToy Sender*". Also a Windows explorer window will open to show the data folder for ShaderToySender in "My Documents". A file "Shaders.zip" will have been copied there. The first thing to do is unzip this file which will will create a "Shaders" folder with example shader files. If you prefer, you can copy "Shaders.zip" to any other convenient folder and unzip from there.

Start *ShaderToySender* using the desktop icon and load any of the example shaders to explore the program.

Operation

ShaderToySender will start for the first time using a default gradient shader. Subsequently you can select any of the other example shaders.

The program allows four texture inputs which can be any image of the formats BMP, TGA, DDS, PNG, JPG. After you have extracted "Shaders.zip", Shadertoy textures are in the TEXTURES folder and sample public images are in the IMAGES folder.

Loading shader files

A number of example shaders are included with the distribution shaders zip file. These can be loaded using the "Shader > Load" menu option or by drag and drop onto the *ShaderToySender* window. The last shader used when you quit the program will be recalled when it starts again.

Controlling the shaders

Mouse coordinates are sent to the shader when the LH button is down. Move around the window to control the shader.

You can also see and test the controls which would appear for a Freeframe plugin with RH mouse click on the window. This will open a controller program which will have all the controls used by that particular shader. A second RH click will hide the controller and another will bring it back etc. The controller is closed when another shader is loaded.

Some shaders look at the mouse button state to determine whether the mouse button is down. To emulate this, increase the value of “X mouse left” or “Y mouse left” to indicate that the left mouse button is down. Then these shaders will react to the X and Y mouse controls.

Creating a shader file

Shader files are created by copy and paste from the code shown by *GLSL Sandbox* or *Shadertoy*. Simply paste the contents into Notepad or a similar text editor and save as a text file. Be careful to avoid line wrap which can break lines of code.

A few shortcut hints might be helpful. Once you have the web shader running, locate the cursor at the very beginning of the code an click once. CTRL-SHIFT-END should then highlight all the code. Copy this to the clipboard. RH click - Copy or CTRL-C. Now paste to the text editor.

It is often useful to have *ShaderToySender* running at the same time. Load a shader file and select “Shader > Edit”. Then "File New" and paste with “RH click - Paste” or “CTRL-V”. Then save the new file. Now you can load it straight away.

If shaders fail to work, first check that that copy/paste has not resulted in line wrap. Any shader requiring audio input might not work and not all pre-defined uniform variables are supported (see below).

Importantly, check the licence terms of the shader authors as detailed below. A link to the website source and credit to the author should be included in the shader source file where possible.

Editing shader files

Shaders can be edited by using the "Shader > Edit" menu option. File types supported are “txt” or “glsl”. The files are opened with whatever program is assigned to open the particular file type. A popular editor is Notepad++.

As you are editing, ShaderToysender will check the file status every half second. if you change and save the file it will be reloaded so that you can immediately see the result of the changes.

If there is an error, a MessageBox will open briefly to show that the shader did not compile. You can keep editing and the file will be monitored until you close it or open another shader.

Debugging shaders

If a shader fails to compile, ShaderToySender opens a console window which shows numbered lines of the shader source. A log of errors follows the source with line numbers for the detected errors.

Shaders are now GLSL "#Version 330" by default based on code for the the official ShaderToy app. As a result, you might see "*deprecated*" warnings which can be ignored.

The console menu close button is disabled or, if selected, it would close the entire application. Refer to the line numbers and edit the shader as necessary. When the error is resolved, the console window is closed.

Debugging for FreeframeGL 2.0

Shaders destined for FreeframeGL 2.0 plugins are translated internally to GLSL "#Version 410" code before the plugin is saved. Errors specific to this version will not be shown within ShaderToySender itself. However, if compilation fails when the plugin is loaded by Resolume, the same console window opens with numbered lines of the translated shader and an error log.

The console will be closed when Resolume is restarted and will not appear if the error is resolved or the plugin is deleted.

For example, some shaders like "SHADERS-7\legend-of-zelda" re-use reserved words, such as "FragColor", which will not work with the translation process. In this case, renaming the variable resolves it. Compare the example with the original on the ShaderToy website (https://www.shadertoy.com/view/4sjGD1).

A common error is implied cast, such as creating an int from float functions, such as "floor(x)". In this case, create a float rather than an int or perform a cast such as int(floor(x)).

Texture sampling for noise functions depends on texture wrapping to be set correctly and Resolume itself does not use texture wrap modes. However, this has been resolved for most shaders. In more complex shaders some functions may still fail for unknown reasons.

Reload

You can also use "Shader > Reload" to load the changed file. The editor can remain open while you load other shaders, so you can edit and compare different shader files.

Reset

Use "Shader > Reset" to completely clear the shader and images. The default shader is then loaded from internal resources of the program. All registry values are also cleared.

The Export menu

Options

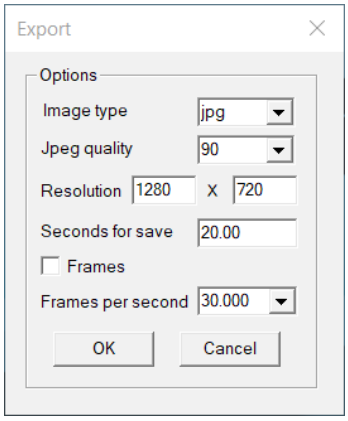


Image type
Choose to save either jpg, bmp or png images.

Jpeg quality
Select the percent quality for jpeg compression. Default is 90.

Resolution
Enter the resolution for the image save. This is independent of the window resolution.

Export duration (Seconds for save / Number of frames)
Enter either the number of seconds for saving a sequence or the number of frames to save. Each is based on the frame rate. For example, if 600 frames are to be saved at 30 fps, the number of seconds will be 20. If 10 seconds is selected instead, the number of frames will be 300 at 30 fps and 600 at 60 fps.

Frames / seconds
Choose between entering the number of frames to save or the number of seconds (with the frame number calculated from the frame rate).

Frames per second
Drop down selection of frame rate. Options are 24, 25, 30, 50, 60. Commonly 30 fps is used.

Select output folder

Browse to select the folder where images will be saved. folder selected.

Open output folder

Opens a file explorer window of the output folder selected.

Save Image

At any time you can save a snapshot of the current frame. A Windows bitmap, jpeg or png image is saved at the current resolution in the output folder that you selected. The file is named after the current shader e.g. : shader_20191120_124009.jpg. Decode this as : 2019 year, 11 month, 20 day, 12 hours, 40 minutes, 09 seconds.

Save image sequence

This option will save a sequence of images, numbered as _0000, _0001, _0002 etc. Each image is saved only after the last has been completed. The rendering is paused during image save and the amount of time will vary depending on the resolution and render speed of the shader and the size and type of image. Rendering is resumed as though only the frame rate time had elapsed. Status is shown in the ShaderToySender window. Press ESCAPE to pause or abort the save.

Delete image sequence

Browse to select any image in the sequence you want to erase.

The Help menu

Documentation

Opens this document.

Check for update

Checks the Spout website to find if there is a more recent version. If there is, you can save the zip file, unzip to a convenient location and run the installer. If the version is current, you will receive a message “ShaderToySender is up to date”.

Diagnostics

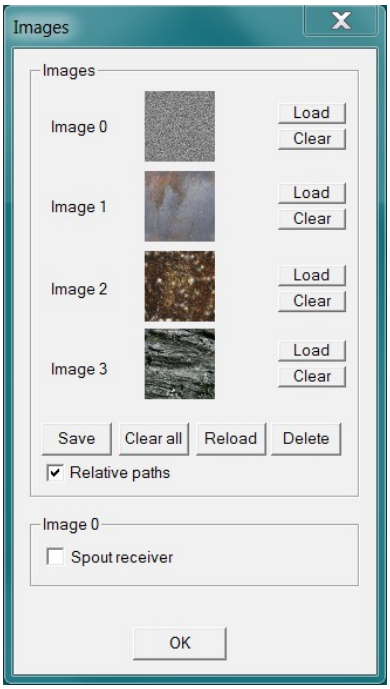
System diagnostics equivalent to the Spout demo programs.

About

Sender information, version and link to the Spout website.

Shader images

Open the Images dialog with “Shader > Images”.

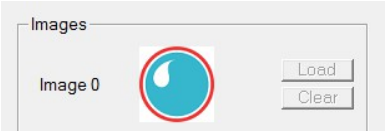


Four textures can be loaded. Select the “Load” button for each and browse to find the required image file. Each image can be individually cleared or loaded again.

To find the images necessary, edit the shader with “Shader > Edit” and examine the comments, or open the url for the shader on the ShaderToy website and scroll down to below the shader code to see what images are used. The names of the images on the ShaderToy website might not be the same, but it is usually easy to recognise which images are required.

Spout receiver

Instead of an image for the first texture, you can use a Spout receiver. This way you can use moving video or graphics from a Spout source, apply a shader to it and receive the output in another program.



Once this option has been selected, the first image is removed and a FreeframeGL plugin will be saved as an effect. So you can create the same result that you see in ShaderToySender by using the host to provide the same input to the plugin.

If you want to save a source plugin that uses Image 0, you need to disable Spout input before you save the plugin. Without the Spout receiver option selected, a prompt reminds you that in order to save an effect plugin, the image has to be cleared.

After the image dialog is closed, you can select any Spout sender that is running by pressing the MIDDLE mouse button. If Spout has been installed, the “SpoutPanel” sender selection dialog will come up and enable you to select a sender.

Because ShaderToySender is a sender itself, you will see the shader sender name in the list. You can't use this though because ShaderToySender would receive from itself and create a feedback loop. A warning is displayed if you do so.

Save

Once the required images are loaded, their image paths can be saved in a file based on the shader name. For example, for a shader file named “my-big-shader.txt”, the image paths are saved in a file named “my-big-shader_images.txt” in the same folder as the shader file. Once images have been saved, they are recalled every time that shader is loaded.

Relative Path

The image paths can be relative to wherever the shader file is located or absolute to wherever each image is located. You can change this option using the “Relative path” checkbox. The default is to save absolute paths, but then you can't move the images. You may prefer the paths to be relative to the shader file if you save images in a folder associated with the shader folder.

Reload

If an image path file has been saved, it can be reloaded using this button and any images loaded will be replaced.

Delete

If an image path file has been saved, but later you find that in fact the shader does not require images, it can be deleted using this button.

Creating a FreeframeGL plugin

If you have found a shader that you want to use in a Freeframe host program independently of *ShaderToySender*, you can create a FreeframeGL plugin directly from the menu without any programming at all. This option replaces the original "*ShaderLoader*" plugin for 32bit host programs.

Saving a plugin

Simply select "Plugin > Save" and select options to save for a 32bit or 64bit host or for FreeframeGL 2.0.

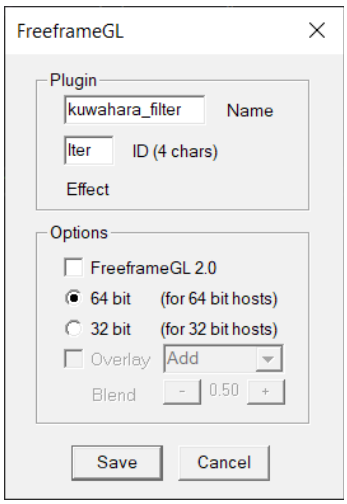
Source and Effect plugins

There are two types of Freeframe plugin, "Source" and "Effect".

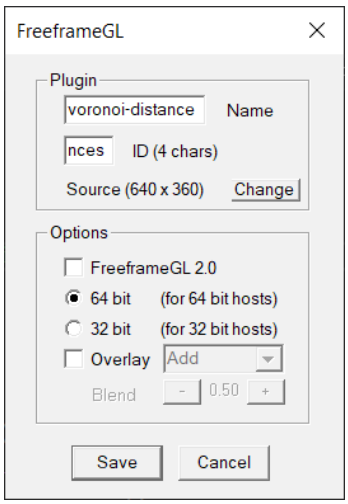
If the plugin generates graphics, it is configured as a *Source* plugin. If you have loaded an image for an effect plugin to see what the effect looks like, it will also be configured as a Source plugin.

If you want it to appear as an effect in the host, clear any images with "Shader > Images > Clear" before you save the plugin.

For an Effect or a Source plugin you will see either :



Effect



Source

The "Overlay" option and the resolution "Change" button are disabled for an effect see more about that below.

Plugin name

Freeframe plugin names are limited to 16 characters and so the shader file name is truncated to this limit. You only need to change this if you want a different name for some reason.

Plugin ID

The Plugin ID is a 4 character identifier which must be unique for each plugin. By default ShaderToySender uses the last 4 characters of the original shader name. This is typically unique, but in some cases it could be the same for different plugins.

You can name the shader files in advance to avoid this problem e.g. my-shader_ID0, my-shader_ID1 etc. But if you are organised and keep a record of the ID's you have used already, you have the option to change the plugin ID before you save it.

Resolution change

Effect plugins always operate at the resolution of the host program, whereas source plugins operate at the resolution selected for ShaderToySender. The selected resolution is shown next to the plugin type. Click “Change” if you want to change it. This opens the Window>Resolution dialog. See more details in the “Shader resolution” section below. The selected resolution remains set and is not restored to the previous values if Cancel is selected.

FreeframeGL 2.0

Choose this option if you want to save plugins for Resolume 7. ShaderToy shaders are translated internally to compatible code. Plugins are 64 bit and Overlay options are not available.

64 bit

Choose this if the host you are saving for is a 64 bit application.

32 bit

Choose this if the host you are saving for is a 32 bit application.

Click “Save” and choose the folder where you want the plugin to be saved.

After you have saved the plugin, it will appear in the Freeframe host under it's name. In Resolume for example, you will find it in the list of effects and you can use it as you would any other effect plugin by dropping it over a cell containing a texture source.

After saving it, any images used by the shader are embedded in the plugin. The original shader and image files are no longer necessary for the plugin to function.

Plugin folders

Resolume recommend that you save them in the 'Extra effects' subfolder in Resolume's user folder in Documents.

The Freeframe convention of using “C:\Program Files (x86)\Common Files\Freeframe” as a generic folder for plugins can be a problem due to permissions with Windows 10. It's best to save them somewhere else and configure the host for that folder.

Example of saving a source or effect plugin

Load an effect shader "..\SHADERS-1\thermal-imaging.txt". The installation shader will also load a penguin image for the shader to show the effect.

Save this shader as a plugin. Of course choose “FreeframeGL 2.0”, “64 bit” or “32 bit”, depending on the host you are using.

But in Resolume you will find it in the list of Sources instead of Effects. Why? - because an image was loaded for the plugin to use for the first texture and that is now embedded in the plugin.

To get this to work as an effect on textures within the host program, simply clear the penguin image before saving the plugin by using "Shader > Images > Clear all". Save the plugin again and over-write the previous one.

Then it will be saved as as an effect plugin, and in Resolume you will find it in the list of Effects instead of the list of Sources. Drag this to a cell containing the media you want it to operate on and you will see the thermal effect.

Overlay effect

You have the option of saving a Source plugin as an “Overlay” on the host image. An example of an overlay is : "..\SHADERS-1\glslnumber.txt".

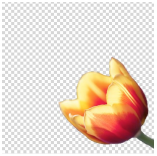
If you check “Overlay” it will be saved as an “Effect” plugin instead of a “Source” and will appear as numbers overlaid on the background image. Change the blend mode to select the result you want. For “Mask” mode, the shader has to be set up for “Alpha” blending. See more about that below.

Overlay mode

Five overlay modes can be selected : Add, Multiply, Screen, Blend, Mask

Blending is applied using OpenGL blending. For the blending functions, the shader result is the foreground and the host image is the background. The equations refer to “s” the source (foreground) and “d” the destination (background). A good reference can be found here.

Foreground image



Background image

Add

```
glBlendFunc(GL_ONE, GL_ONE);
(s*1) + (d*1)
```

Multiply

```
glBlendFunc(GL_ONE, GL_DST_COLOR);
(s*1) + (d*d)
```

Screen

```
glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_COLOR);
(s*1) + (d*(1-s))
```

Blend and Mask

```
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
(s*sA) + (d*(1-sA)) (alpha mask)
(s*amount) + (d*(1-amount)) (blend)
```

Blend
```
(s*amount) + (d*(1-amount))
```

Source and destination are blended by a proportion. If the proportion is 0.5, both are blended by equal amounts. If the proportion is 0.8, the shader result is blended by 0.8 and the background by 0.2 etc.

Mask
```
(s*sA) + (d*(1-sA))
```

For alpha masking, the proportion is replaced by the alpha value. Where the source (s) is the shader and "sA" (alpha), 0 is transparent and 1 fully opaque. Where parts of the shader result are transparent (alpha 0), the background image shows through. Where parts are opaque (alpha 1), the shader result replaces the background. Detailed examples are described further below.

Changing an "Effect" to an "Overlay" plugin

Using the "Overlay" option is most useful where the effect itself is really an overlay. An example is "..\SHADERS-5\snow-storm.txt". The shader generates the snow particles and mixes them with the incoming host texture.

Normally you would use an image to see what the result looked like, then clear the images to save the plugin as an effect and use the host texture as the source for the shader.

Keep in mind that an effect operates as the full resolution of the host, so if you have a project set at 1920x1080 for example, the shader will process every pixel. Most effect shaders are relatively simple but you might find that some more complex effects increase GPU load and reduce fps. Complex effects are relatively infrequent and it is best to avoid these.

However, for "Source" and "Overlay" plugins you have control over the resolution that the shader uses (see more detail below), so by converting from an "Effect" to an "Overlay" plugin you have control over the shader resolution and performance of the plugin.

By using "Overlay", together with some modification of the shader code, you can reduce resolution of the shader to increase performance without affecting the resolution of the host background image.

In this case, all the host texture is used for is to mix with the result of the shader, so it is not actually needed for the shader to work and you can remove that from the shader code. To do this you will need to edit the shader file.

If you edit the shader you will see :

```
fragColor = mix(texture(iChannel0, uv), vec4(1.0), vec4(val));
```

That shows the mixing function which what you want to change.

Have another look at the shader code. The line above is commented out :

```
//fragColor = vec4(vec3(val), 1.0);
```

This is the result of the snow particles, so you can edit the file as so :

```
fragColor = vec4(vec3(val), 1.0);
// fragColor = mix(texture(iChannel0, uv), vec4(1.0), vec4(val));
```

Save the file and then save the plugin again. Now it will be configured as a “Source” and you have the option to save as an “Overlay” (use Blend option “Add” because the background is black). Now the plugin will be configured as an “Effect” and the snow particles overlaid on the background image.

What is happening is that because the shader is generating the snow graphics it becomes a “Source” when the mix function is removed. When you save as an “Overlay effect”, the mixing is done by the plugin instead of the shader.

Mask overlay mode (Alpha masking)

Alpha masking depends on the shader creating the transparency for the overlay. Transparent (Alpha 0) and opaque (Alpha 1).

ShaderToy shaders are not usually designed for alpha transparency, so this option is most useful for people developing their own shaders or modifying those sourced from ShaderToy.

Firstly, it is important to know that many ShaderToy shaders generate rgb graphics without alpha or alpha set to zero. *If you see a black screen, this could be the reason*. It's simple to fix by adding alpha with a value of 1 to the output.

For example some shaders specify zero for alpha. Just change this to 1.0 :

```
vec3 col = vec3(1.0, 0.0, 0.0); // or something rgb
// fragColor = vec4(col, 0.0); // the shader specifies zero
fragColor = vec4(col, 1.0); // change the alpha value to 1.0 output
```

Some shaders specifically generate rgb, such as this solid red for example :

```
fragColor.rgb = vec3(1.0, 0.0, 0.0);
```

You can simply add alpha to the output.

```
FragColor.a = 1.0;
```

For alpha masking, parts of the output will show and other parts reveal the background image. Set alpha to 1 where you want it to show and 0 where you want the background revealed, then save the plugin as an Overlay with blend mode set to “Alpha”.

A simple example is “..\SHADERS-6\pass-through-alpha.txt”

```
//
// ShaderToySender
//
// Example of using alpha for an Overlay plugin
// For the background image to show through, set alpha to 0 (transparent)
// For the shader to show on top, set alpha to 1 (opaque)
// To combine the two, set alpha to a fraction (shader/background)
//
void mainImage( out vec4 fragColor, in vec2 fragCoord ) {

    vec2 uv = fragCoord.xy/iResolution.xy;
    vec3 c0 = vec3(0., 0., 1.);
```

```
    if (uv.x < iMouse.x/iResolution.x)
    {
        // Left hand side of the image is transparent (alpha = 0)
        // The background image will show through
        fragColor = vec4(c0, 0.);
    }
    else
    {
        // Right hand side of the image is opaque (alpha = 1)
        // The background image will be overlaid by the shader (blue)
        fragColor = vec4(c0, 1.);
    }

}
```

Here there is a solid blue output where the alpha value is set to 0 on one side and 1 on the other. Wherever alpha is 0, the underlying image shows through. You can slide this across with Mouse X.

An example of a black and white mask adapted from a ShaderToy shader is "..\SHADERS-6\noise-holes.txt". Edit the shader file and scroll to the end to see the changes for an alpha mask.

```
    if (noise < CUTOFF){
       // white wall
       col = vec3(1., 1., 1.); // white background
       alpha = 0.; // transparent

    }
    else {
      alpha = 1.; // black blobs
    }
```

A more complex example is "..\SHADERS-6\slime-borders.txt". Scroll to the end to see the changes. The top part is the centre and the bottom part is the slime border effect :

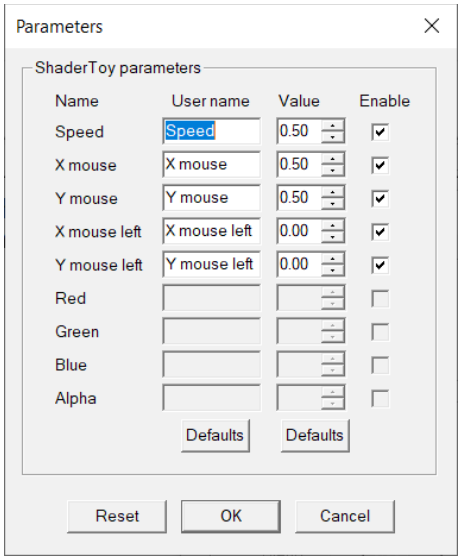
```
    if (k.g <= 0.4)
    {
       float m = 1.0-clamp(k.g,0.0,0.5);
       float f = mod( floor(12.0*uv.x) + floor(12.0*uv.y), 2.0);
       float col = 0.3 + 0.2*f*1.0;
       k.x = k.y = k.z = col*m;
       k.rb *= 1.2;
       k.a = 0.; // transparent for an alpha mask overlay
    }
    else
    {
       float f = mod( floor(12.0*uv.x) + floor(12.0*uv.y), 2.0);
       vec4 _col = 0.2*f*vec4(0.175);
       k += _col;
       k.a = 1.; // opaque for an alpha mask overlay
    }
```

If you save the plugin as an "Overlay" with blend mode set to "Mask" you will see the mask effect.

If you want to the original output, change **"k.a = 0.;"** to **"k.a = 1.;"** and save as a "Source" plugin.

Shader parameters

The “Plugin > Parameters” dialog allows you customize your plugin by changing any of the parameters the shader might be using. You can change the parameter name, starting value and enable or disable it.



Normally, you don't have to worry about this. In fact the majority of shaders will only have “Speed” control. But other shaders use mouse position, usually to control orientation, and you might want to fine tune the parameters that appear in the plugin by selecting the ones you want and changing their names.

Defaults

If you have made changes and want to start over, select “Defaults” for either the parameter names or values.

Reset

If changes have been made and the plugin dialog closed, only the parameters that have been enabled will be available. If the controls are opened with RH click or if you open the parameter dialog again, the parameters that have been disabled will not be listed. If you want to start again, click “Reset”.

How to use the parameters

ShaderToy has a fixed set of uniform variables that the shader can use. Details are explained further below, but the ones available to the plugin are :

Speed	iTime	Controls the speed of shader animation (current time in seconds)
X mouse	iMouse.x	Controls mouse X coordinate
Y mouse	iMouse.y	Controls mouse Y coordinate
X mouse left	iMouse.z	Controls mouse X coordinate when the left button is pressed
Y mouse left	iMouse.w	Controls mouse Y coordinate when the left button is pressed
Red	inputColour.r	Can be used for any purpose within the shader
Green	inputColour.g	
Blue	inputColour.b	
Alpha	inputColour.a	

To give you an idea of how these parameters can be changed, we will step though an example that includes mouse control.

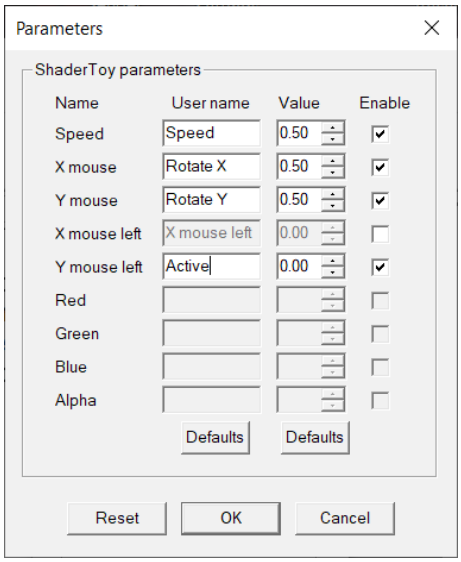
Load "..\SHADERS-3\voxel-edges.txt". An image will also be loaded for shader to show the effect. Within *ShaderToySender* you can change the orientation with the mouse. Move left/right, up/down when the left button is down.

RH click will bring up the controls that the plugin will use. But if you try “X mouse” and “Y mouse” nothing will happen. That is because the shader is looking for the Y mouse coordinate when the left button is pressed (iMouse.w) to indicate when the mouse button is down.

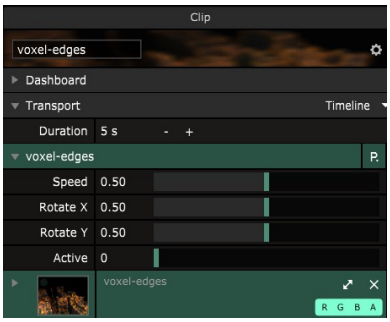
On the ShaderToy website this works as intended. But the plugin mouse controls are isolated from the mouse itself, so you need to increase the value of “Y mouse left” from zero to indicate to the shader that the mouse left button is down. Then you will find that the Mouse X and Y controls work as expected.

This can be confusing and so you might want to change the parameter names to make it easier for the user to understand what to do. So let's do that with the Parameter dialog.

Firstly the “X mouse left” control is not used, so disable it with the checkbox at right. Change the name of the “Y mouse left” parameter to something that indicates to the user whether the controls will work. In this case “Active” will do. Also it might be better to change the name of the mouse controls because we are not actually using a mouse. Here “Rotate X” and “Rotate Y” are appropriate. Click OK when done.



Now when you save the plugin, it will have the controls and names that you have set up instead of the defaults. In Resolume for example it will appear as below :



Now slide “Active” to the right so that it is off zero and the two rotate controls will work as expected.

Of course, if you are familiar with shader programming, you can change the shader source file to remove “iMouse.w” and it will have the same result.

The shader will no longer be suitable for the ShaderToy website, but that's not a problem for a plugin. Examine the shader source and you will see a suggested modification

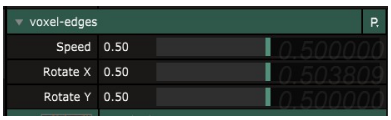
```
// Optionally remove this line for ShaderToySender because
// we do not have to disable mouse control when the button is up
if( iMouse.w<=0.00001 ) mo=vec2(0.0);
```

Edit this line as so :

```
// if( iMouse.w<=0.00001 ) mo=vec2(0.0);
```

Save the shader file and “Reload”. Now you will find that RH click controls for Mouse X and Y will work without having to change “Y mouse left” as before.

The parameter names you entered before will now have been reset, so you have to enter them again. Open the Parameter dialog again and uncheck both “X mouse left” and “X mouse left” because now neither are necessary. Rename X and Y mouse to Rotate, close the dialog and save the plugin once more, overwriting the previous one. Rotate X and Y will work straight away.



Advanced use of the parameters

You can go further to modify shaders for mouse control. This is possible where shaders define constant values that can be made variable.

One example is “..\SHADERS-5\metaball-kaleidoscope.txt”.

You will see in the shader file :

```
// ---- change here ----
const float USE_KALEIDOSCOPE = 1.0;
const float NUM_SIDES = 6.0;
```

“USE_KALEDOSCOPE” disables the kaleidoscope effect and shows moving metablobs instead. “NUM_SIDES” controls the number of sides and complexity of the kaleidoscope. These are constants, so we make them variable for control with the mouse.

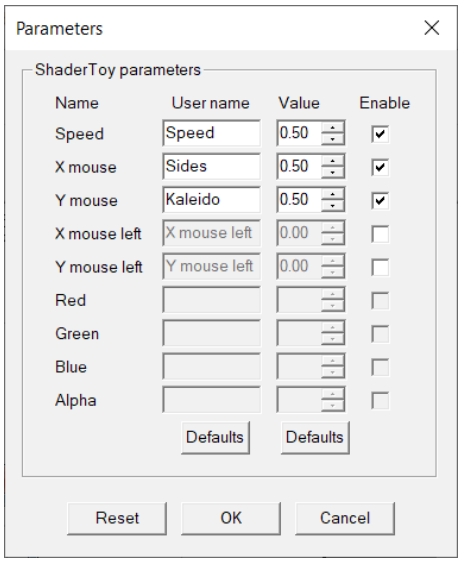
Change from USE_KALEIDOSCOPE” from “const” to variable and use a float in Main for NUM_SIDES.

```
// ---- change here ----
float USE_KALEIDOSCOPE = 1.0;
// Use a local float variable in main for the number of sides
```

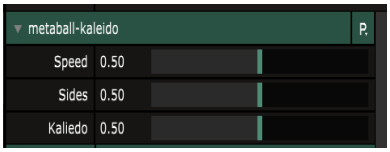
Now further down, change the code so that these parameters can be varied with the mouse :

```
void mainImage( out vec4 fragColor, in vec2 fragCoord )
{
    vec2 ratio = iResolution.xy / min( iResolution.x, iResolution.y );
    vec2 uv = ( fragCoord.xy * 2.0 - iResolution.xy ) / min( iResolution.x, iResolution.y );

    // ShaderToySender changes for mouse control
    vec2 mouse = iMouse.xy/ iResolution.xy; // normalised mouse coords 0-1
    float NUM_SIDES = mouse.x*12.0; // number of sides 0-12
    if(mouse.y < 0.5) USE_KALEIDOSCOPE = 0.0; // Disable if Y control is < 0.5

    // --- Kaleidoscope ---
    uv = mix( uv, Kaleidoscope( uv, NUM_SIDES, iTime * 0.5 ), USE_KALEIDOSCOPE )
```

Save the shader file, and change the parameter names to match :



Now when you save the plugin and it will appear in Resolume with the custom names.



Now you can change the number of sides and enable/disable the kaleidoscope. You might want to alter the shader code further to protect against zero sides if you like.

Experiment with more ShaderToy shaders, or one of your own that you want to adapt and save as a plugin.

Remember that you can use the four values of the “inputColour” uniform to change any float value in the shader.

Parameter values

One more thing that was mentioned briefly before is changing the starting value of each parameter.

Normally you will not need to change these. Most defaults are 0.5 which is the middle of the range 0.0 – 1.0. But the defaults for “X mouse left” and “Y mouse left” are zero and 1.0 for Alpha. If you want to use these as general purpose variables within the shader you might need to change the starting value.

Let's look at an example.

“..\SHADERS-6\brcosa.txt”

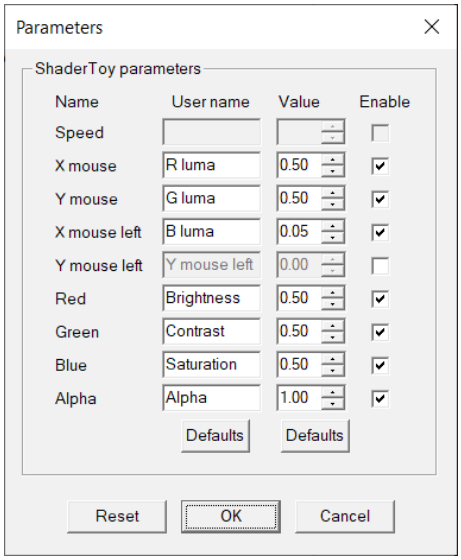
This is a shader that adjusts the brightness, contrast, saturation and alpha of an image. It is based on a Processing example which allows three of these, brightness, contrast and saturation. The shader is extended to pass in the average luminance and the alpha value.

When you load the shader it has a slight yellow tinge. That is because the Blue component of the average luminance is passed in using “X mouse left” and the default value is zero instead of 0.5. To see what is happening bring up the Parameter dialog :

Here we can change the names and default values of the parameters.

Name	User name	Value
X mouse	R luma	0.5
Y mouse	G luma	0.5
X mouse left	B luma	0.5
Red	Brightness	0.5
Green	Contrast	0.5
Blue	Saturation	0.5
Alpha	Alpha	1.0

The only Value to change is “X mouse left” from 0.0 to 0.5. “Y mouse left” is unused, so disable it.



Click OK and you will see the yellow tint change to neutral because the value of “X mouse left” has been changed.

Changing parameter values

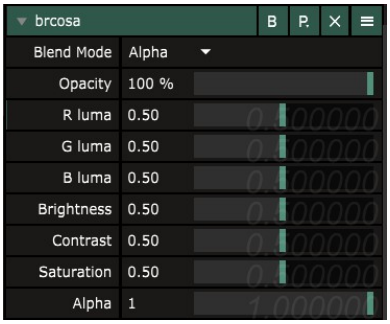
The default values of the parameters can be adjusted with the up/down controls for each parameter. These defaults are used by the plugin when it starts.

You can also adjust the values before changing the names by using the Controller.

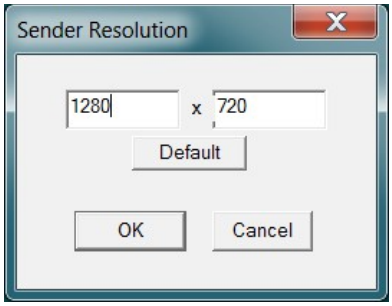
RH click before you use the Parameter dialog and adjust the values as you require, in this case adjust “X mouse left” from 0.00 to 0.50, and then open the Parameter dialog. The default values will have been set.

Now you can save the plugin. If you retain the example image, the plugin will be saved as a source. Before saving it, open the Shader Images dialog and “Clear all” to clear the images. Then it will be saved as an effect.

This is how it will appear in Resolume :



Shader resolution



For a Source or Overlay plugin, the resolution set with "Window > Resolution" is saved within the plugin and used by the shader. Effect shaders always work at the resolution of the host.

For example, if the resolution is set to 640x360, these dimensions are embedded in the source plugin which will then render at that resolution. This has no effect on the final display which comes out at whatever size the host is producing.

Because the source plugin render resolution is independent of the host program you can use high resolutions for the the host output without affecting the speed of the plugin.

Performance can be significantly improved, particularly for shaders that require a lot of processing. In fact, for some complex shaders, you might find it necessary to use even less than the default resolution of 640x360.

The fps gained by changing resolution can be seen in the *ShaderToySender* window. A plugin will not achieve the same performance, but this is a guide to the resolution that can be used.

Configuration

Any changes you make to options for relative image paths, resolution or topmost window are saved in an initialization file “shadertoysender.ini” which is loaded when the program is started to restore the previous settings.

Shader uniform variables

Although *GLSL Sandbox* and *ShaderToy* have different naming conventions, either of these can be used. To distinguish the file types, the exact string "uniform float time;" has to be present in a *GLSL Sandbox* file.

ShaderToy

ShaderToySender uses the same concept as the ShaderToy website. There is a single vertex shader and a set of pre-defined uniform variables for the fragment shader. These are added internally when the shader file is loaded.

There is no audio input, and multipass is not supported so many of the shaders on the websites will not work. Nevertheless, if you choose shaders with a single “Image” shader file, a large number of them can be used either as-is or with minor modification. Mouse interaction is mapped to Freeframe parameters and controls that the host provides as described above.

GLSL Sandbox

```
uniform float time; // Current time in seconds
uniform vec2 mouse; // Normalized mouse position. (between 0.0 and 1.0)
uniform vec2 resolution; // Screen (Viewport) resolution.
uniform vec2 surfaceSize; // size of the visible area of the virtual surface.
uniform vec2 surfacePosition; // position of the current fragment on the virtual surface
uniform sampler2D backbuffer; // sampler for backbuffer
uniform sampler2D texture; // sampler for texture
```

In order to allow for multiple input textures, the following uniform variable names are used, equivalent to the ShaderToy "iChannel" uniforms :

```
uniform sampler2D tex0;
uniform sampler2D tex1;
uniform sampler2D tex2;
uniform sampler2D tex3;
```

Only two input textures are supported for a plugin. Note that “backbuffer” is not supported but is mapped to the first input texture instead. If a shader refers to the backbuffer it might work with the addition of in input texture but will not have the feedback effects that could be intended.

ShaderToy uniforms

```
uniform float iGlobalTime; // (discontinued) current time (in seconds)
uniform float iTime; // current time (in seconds)
uniform float iTimeDelta; // time elapsed since the last time check
uniform float iFrame; // frame number
uniform float iFrameRate; // frame rate (1 / iTimeDelta)
uniform float iSampleRate; // audio sample rate (not used)
uniform vec4 iMouse; // xy are pixel coords (Left button down). zw are the click pixel.
uniform vec3 iResolution; // the rendering resolution (in pixels)
uniform vec4 iDate; // (year, month, day, time in seconds)
uniform float iChannelTime[4]; // channel playback time (in seconds)
uniform vec3 iChannelResolution[4]; // channel resolution (in pixels)
uniform sampler2D iChannel0; // sampler for input texture 0.
uniform sampler2D iChannel1; // sampler for input texture 1.
uniform sampler2D iChannel2; // sampler for input texture 2.
```

uniform sampler2D iChannel3; // sampler for input texture 3.

Additional uniform

uniform vec4 inputColour; // (red, green, blue, alpha)

This is added internally for a ShaderToy file, but for GLSL Sandbox it has to be entered into the file itself. The four values can be used for any purpose.

User uniforms

User uniforms are a good way to develop a shader which is independent from the ShaderToy built in uniforms.

Start with the GLSL sandbox uniforms which are always updated by the program or the Freeframe plugin. Then add any number of *float* uniforms which show as a slider control with range of 0.0 to 1.0. There is also a *bool* uniform type with values true or false and this shows as a checkbox control.

Default values are 0.5 for floats and false for bool. They don't show up in the Parameters dialog, but they are already named as you require and in the order you want them to appear in the plugin. Default values can be set using the ShaderToySeneder controls. RH click to open the controls and set the values as you wish. Then these are saved as starting values when the plugin loads.

There is a brief example in "SHADERS7/checkbox.txt"

Licence

ShaderToy

Shaders from the ShaderToy website, are licensed under Attribution-NonCommercial-ShareAlike 3.0 (https://creativecommons.org/licenses/by-nc-sa/3.0/deed.en_US) unless otherwise specified. Where specifically indicated, the licencing terms of the author must be observed.

ShaderToySender

ShaderToySender is Licenced with the LGPL3 license. Copyright (C) 2016-2019. Lynn Jarvis, Leading Edge. Pty. Ltd.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details. You will receive a copy of the GNU Lesser General Public License along with this program. If not, see http://www.gnu.org/licenses/.

Image loading uses the SOIL image library with public domain licence.
Image saving uses the TOOJPEG encoder with zlib License compatible with GPL.

Freeframe

The plugins generated by ShaderToySender use the Freeframe 1.6 and Freeframe 2.0 libraries from the Resolume fork of the FFGL repository.

FreeFrame is an open-source cross-platform real-time video effects plugin system. It provides a framework for developing video effects plugins and hosts on Windows, Linux and Mac OSX. Copyright (c) 2006 www.freeframe.org. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
* Neither the name of FreeFrame nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.